

Лабораторная работа №13

Архитектура компьютера

Содержание I

- Володина Алиса Алексеевна

- Володина Алиса Алексеевна
- НКАбд-01-25

- Володина Алиса Алексеевна
- НКАбд-01-25
- Российский университет дружбы народов им. П. Лумумбы

- Володина Алиса Алексеевна
- НКАбд-01-25
- Российский университет дружбы народов им. П. Лумумбы
- 1032253521

Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов

- 1 Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: - `-iinputfile` — прочитать данные из указанного файла; - `-ooutputfile` — вывести данные в указанный файл; - `-р`шаблон — указать шаблон для поиска; - `-C` — различать большие и малые буквы; - `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-р`.

- 2 Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.

- 3 Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до n (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).

- 4 Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`).

Bash (Bourne Again Shell) — командная оболочка Unix для работы в терминале и автоматизации задач через скрипты. Поддерживается в Linux, Unix и Windows (WSL).

Основные возможности: Обработка простых и сложных команд (конвейер |, перенаправление <, >, », условные операторы) Расширенный ввод: автодополнение команд и файлов, история команд Создание и запуск скриптов (последовательности команд с циклами, условиями, функциями)

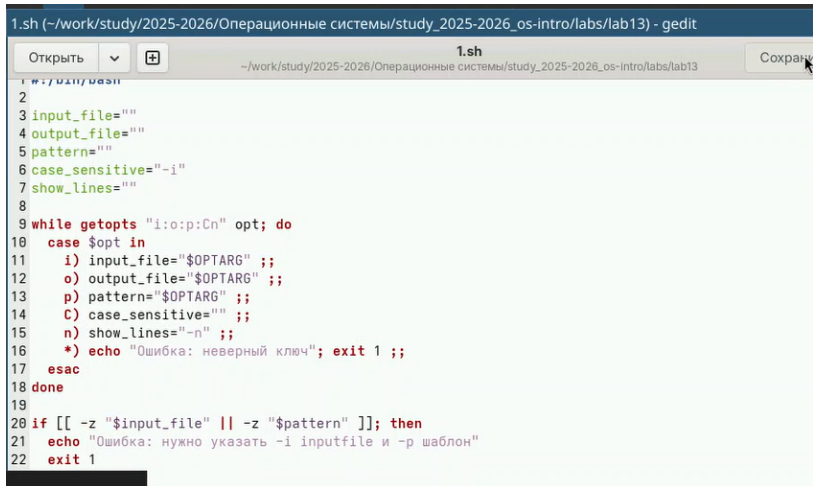
Выполнение лабораторной работы

создадим файл дл первого командного файла и перейдем к редактированию

```
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ touch 1.sh  
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ gedit 1.sh
```

Рисунок 1: создание и редактирование файла

Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: - `-iinputfile` — прочитать данные из указанного файла; - `-ooutputfile` — вывести данные в указанный файл; - `-rшаблон` — указать шаблон для поиска; - `-C` — различать большие и малые буквы; - `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-r`. (рис. 2). (рис. 3).



The screenshot shows a terminal window with a gedit editor. The title bar indicates the path: 1.sh (~/.work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13) - gedit. The editor contains a shell script with the following content:

```
1 #!/bin/bash
2
3 input_file=""
4 output_file=""
5 pattern=""
6 case_sensitive="-i"
7 show_lines=""
8
9 while getopts "i:o:p:Cn" opt; do
10     case $opt in
11         i) input_file="$OPTARG" ;;
12         o) output_file="$OPTARG" ;;
13         p) pattern="$OPTARG" ;;
14         C) case_sensitive="" ;;
15         n) show_lines="-n" ;;
16         *) echo "Ошибка: неверный ключ"; exit 1 ;;
17     esac
18 done
19
20 if [[ -z "$input_file" || -z "$pattern" ]]; then
21     echo "Ошибка: нужно указать -i inputfile и -p шаблон"
22     exit 1
```

Рисунок 2: первый командный файл


```
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ ./1.sh -i test.txt -p hello -C -n -o out.txt  
Результат сохранён в out.txt  
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ |
```

Рисунок 3: работа командного файла

- 2 Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.

```
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ touch 2.c  
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ gedit 2.c
```

Рисунок 4: создание и редактирование файла 2.c

```
1 #include <stdlib.h>
2 #include <stdio.h>
3
4 int main () {
5     int num;
6     printf("Введите число");
7     scanf("%d", &num);
8
9     if (num>0)
10         exit(1);
11     else if (num<0)
12         exit(2);
13     else
14         exit(0);
15 }
```

```
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ touch 2.sh  
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ gedit 2.sh
```

Рисунок 6: создание и редактирование файла 2.sh

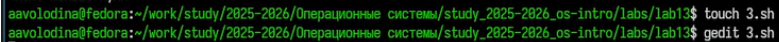
```
1 #!/bin/bash
2 ./2
3 result=$?
4
5 case $result in
6     0) echo "Число равно нулю" ;;
7     1) echo "Число больше нуля" ;;
8     2) echo "Число меньше нуля" ;;
9     *) echo "Неизвестный код" ;;
10 esac
```

Рисунок 7: код 2 командного файла

```
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ gcc 2.c -o 2
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ chmod +x 2.sh
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ ./ 2.sh
bash: ./: Это каталог
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ ./2.sh
Введите число: 5
Число больше нуля
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ ./2.sh
Введите число: 0
Число равно нулю
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ ./2.sh
Введите число: -1
Число меньше нуля
```

Рисунок 8: работа 2 командного файла

- 3 Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до n (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).



```
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ touch 3.sh
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ gedit 3.sh
```

Рисунок 9: создание 3 командного файла

```
1 #!/bin/bash
2
3 if [[ $# -ne 2 ]]; then
4     echo "Использование: $0 --create N или $0 --remove N"
5     exit 1
6 fi
7
8 mode=$1
9 N=$2
10
11 if [[ "$mode" == "--create" ]]; then
12     for i in $(seq 1 $N); do
13         touch "$i.tmp"
14     done
15     echo "Создано $N файлов"
16 elif [[ "$mode" == "--remove" ]]; then
17     for i in $(seq 1 $N); do
18         rm -f "$i.tmp"
19     done
20     echo "Удалено $N файлов"
21 else
22     echo "Неверный режим. Используйте --create или --remove"
23 fi
```

```
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ chmod +x 3.sh  
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ ./3.sh --create 5  
Создано 5 файлов  
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$
```

Рисунок 11: работа 3 скрипта(создание файлов)

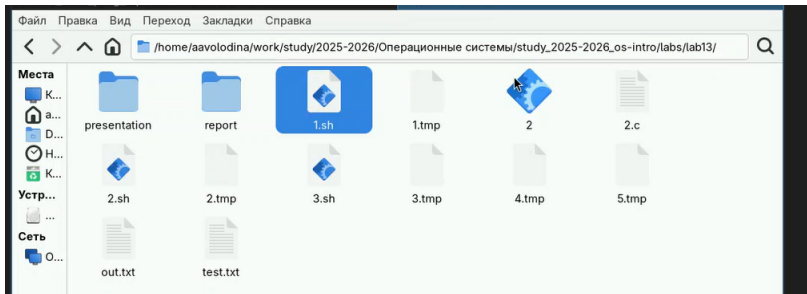


Рисунок 12: проверка созданных файлов

```
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ ./3.sh --remove 5  
Удалено 5 файлов  
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ |
```

Рисунок 13: работа 3 скрипта(удаление файлов)

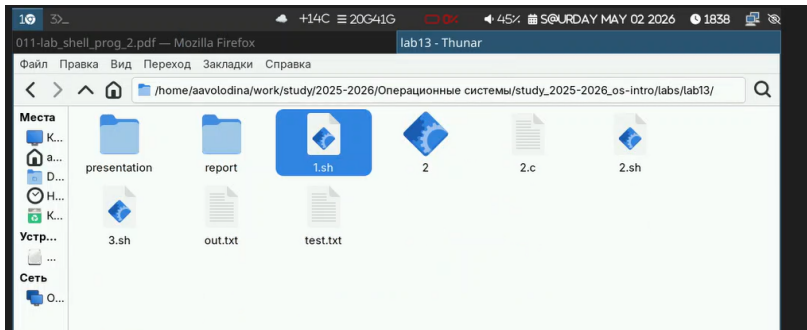
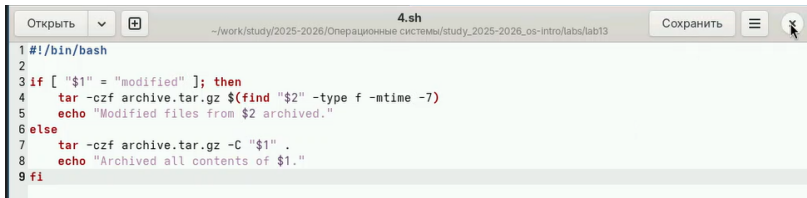


Рисунок 14: проверка удаленных файлов

- 4 Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`). (рис. 15). (рис. 16).



```
1 #!/bin/bash
2
3 if [ "$1" = "modified" ]; then
4     tar -czf archive.tar.gz $(find "$2" -type f -mtime -7)
5     echo "Modified files from $2 archived."
6 else
7     tar -czf archive.tar.gz -C "$1" .
8     echo "Archived all contents of $1."
9 fi
```

Рисунок 15: код 4 командного файла

```
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ mkdir test_folder
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ echo "test" > test_folder/file.txt
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ ./4.sh test_folder
Archived all contents of test_folder.
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$ tar -xzf archive.tar.gz
aavolodina@fedora:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro/labs/lab13$
```

Рисунок 16: работа 4 командного файла

Я изучила основы программирования в оболочке ОС UNIX. Научилась писать более сложные командные файлы с использованием логических управляющих конструкций и циклов

Контрольные вопросы

- 1) Предназначение getopt — разбор параметров командной строки в скриптах (например, -a, -f файл), автоматическая обработка флагов и их аргументов.

Контрольные вопросы

- 1) Предназначение `getopts` — разбор параметров командной строки в скриптах (например, `-a`, `-f` файл), автоматическая обработка флагов и их аргументов.
- 2) Метасимволы (`*`, `?`, `[]`) используются для генерации имён файлов (шаблонов подстановки) — оболочка заменяет их списком подходящих файлов.

Контрольные вопросы

- 1) Предназначение `getopts` — разбор параметров командной строки в скриптах (например, `-a`, `-f` файл), автоматическая обработка флагов и их аргументов.
- 2) Метасимволы (`*`, `?`, `[]`) используются для генерации имён файлов (шаблонов подстановки) — оболочка заменяет их списком подходящих файлов.
- 3) Операторы управления действиями: `&&` (И), `||` (ИЛИ), `;` (последовательное выполнение), `&` (фон), `|` (конвейер), `()` (подоболочка), а также `,`, `'`, `"` для экранирования/группировки.

Контрольные вопросы

- 1) Предназначение `getopts` — разбор параметров командной строки в скриптах (например, `-a`, `-f` файл), автоматическая обработка флагов и их аргументов.
- 2) Метасимволы (`*`, `?`, `[]`) используются для генерации имён файлов (шаблонов подстановки) — оболочка заменяет их списком подходящих файлов.
- 3) Операторы управления действиями: `&&` (И), `||` (ИЛИ), `;` (последовательное выполнение), `&` (фон), `|` (конвейер), `()` (подоболочка), а также `,`, `'`, `"` для экранирования/группировки.
- 4) Операторы прерывания цикла: `break` (выход из цикла), `continue` (переход к следующей итерации).

Контрольные вопросы

- 1) Предназначение `getopts` — разбор параметров командной строки в скриптах (например, `-a`, `-f` файл), автоматическая обработка флагов и их аргументов.
- 2) Метасимволы (`*`, `?`, `[]`) используются для генерации имён файлов (шаблонов подстановки) — оболочка заменяет их списком подходящих файлов.
- 3) Операторы управления действиями: `&&` (И), `||` (ИЛИ), `;` (последовательное выполнение), `&` (фон), `|` (конвейер), `()` (подоболочка), а также `,`, `'`, `"` для экранирования/группировки.
- 4) Операторы прерывания цикла: `break` (выход из цикла), `continue` (переход к следующей итерации).
- 5) `true` всегда возвращает код 0 (успех), `false` — код 1 (ошибка).
Используются для бесконечных циклов, проверки логики и задержек.

Контрольные вопросы

- 1) Предназначение `getopts` — разбор параметров командной строки в скриптах (например, `-a`, `-f` файл), автоматическая обработка флагов и их аргументов.
- 2) Метасимволы (`*`, `?`, `[]`) используются для генерации имён файлов (шаблонов подстановки) — оболочка заменяет их списком подходящих файлов.
- 3) Операторы управления действиями: `&&` (И), `||` (ИЛИ), `;` (последовательное выполнение), `&` (фон), `|` (конвейер), `()` (подоболочка), а также `,`, `'`, `"` для экранирования/группировки.
- 4) Операторы прерывания цикла: `break` (выход из цикла), `continue` (переход к следующей итерации).
- 5) `true` всегда возвращает код 0 (успех), `false` — код 1 (ошибка).
Используются для бесконечных циклов, проверки логики и задержек.
- 6) `if test -f mans/i.s, mans/i.s` (например, `man1/l.s.1`) обычным файлом (`-f`).

Контрольные вопросы

- 1) Предназначение `getopts` — разбор параметров командной строки в скриптах (например, `-a`, `-f` файл), автоматическая обработка флагов и их аргументов.
- 2) Метасимволы (`*`, `?`, `[]`) используются для генерации имён файлов (шаблонов подстановки) — оболочка заменяет их списком подходящих файлов.
- 3) Операторы управления действиями: `&&` (И), `||` (ИЛИ), `;` (последовательное выполнение), `&` (фон), `|` (конвейер), `()` (подоболочка), а также `,`, `'`, `"` для экранирования/группировки.
- 4) Операторы прерывания цикла: `break` (выход из цикла), `continue` (переход к следующей итерации).
- 5) `true` всегда возвращает код 0 (успех), `false` — код 1 (ошибка).
Используются для бесконечных циклов, проверки логики и задержек.
- 6) `if test -f mans/i.s, mans/i.s` (например, `man1/l.s.1`) обычным файлом (`-f`).
- 7) `while` выполняет цикл, пока условие истинно (код возврата 0), `until` — пока условие ложно (код возврата не 0). То есть `until` работает до